Свёрточная нейронная сеть. Часть 2

>

Теперь, когда у нас достаточно знаний об операции свёртки и понятен принцип, согласно которому свёрточная нейросеть может обнаружить интересующий нас объект, самое время перейти к изучению общей архитектуры сети.

Архитектура свёрточной нейросети

Подготовка данных

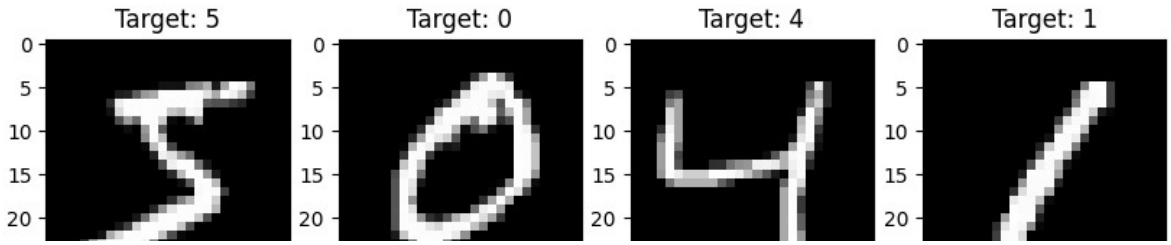
Возьмем уже знакомый нам датасет рукописных цифр MNIST и создадим свёрточную нейросеть-классификатор в библиотеке Keras.

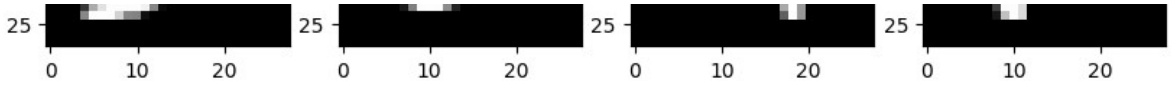
```
from tensorflow import keras

(X_train, y_train), (X_test, y_test) = keras.datasets.mnist.load_d

fig, axes = plt.subplots(1, 4, figsize = (10, 3))

for ax, image, label in zip(axes, X_train, y_train):
    ax.imshow(image, cmap = 'gray')
    ax.set_title(f'Target: {label}')
```





На изображениях выше хорошо видно, что каждая цифра состоит из расположенных под разными углами контуров. Именно эти контуры мы и постараемся запомнить, создав в процессе обучения модели фильтры, похожие на фильтры Собеля.

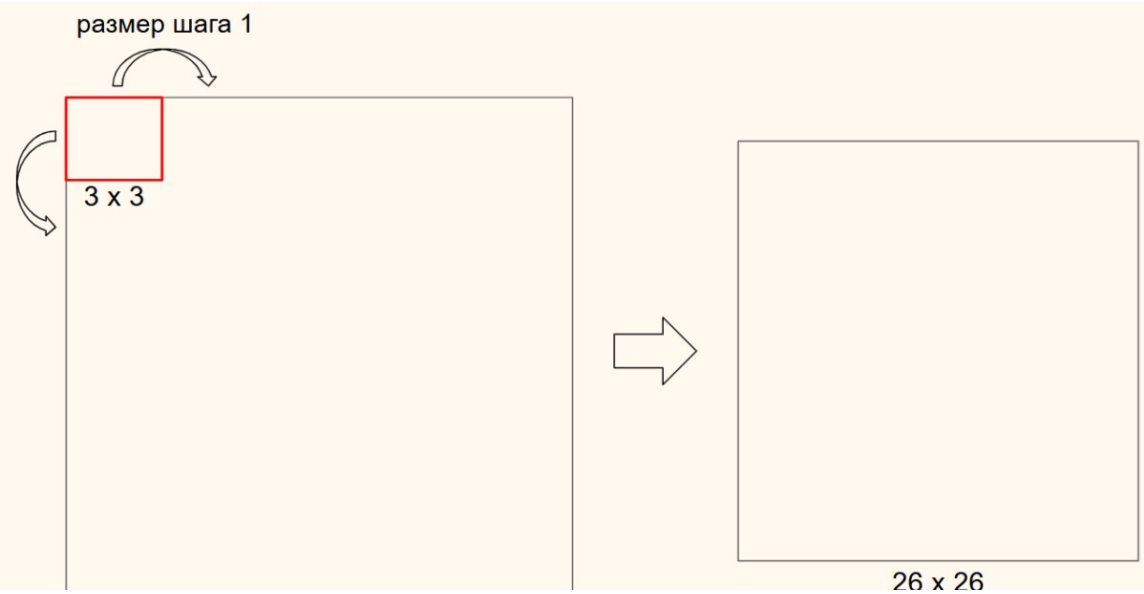
```
# приведем изображения к диапазону от -0,5 до 0,5
X_train = (X_train / 255) - 0.5
X_test = (X_test / 255) - 0.5
```

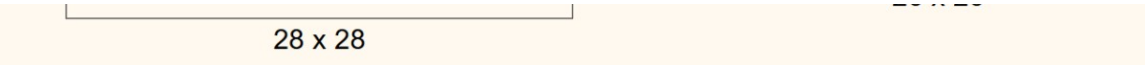
Свёрточный слой и подвыборка

Начнем создавать **свёрточный слой** (convolutional layer):

- мы знаем, что каждая картинка имеет размерность 28 x 28 пикселей;
- пусть фильтр будет размером 3 x 3;
- шаг оставим равным единице; а
- отступ пусть будет нулевым.

В этом случае размерность карты признаков составит 26 x 26.



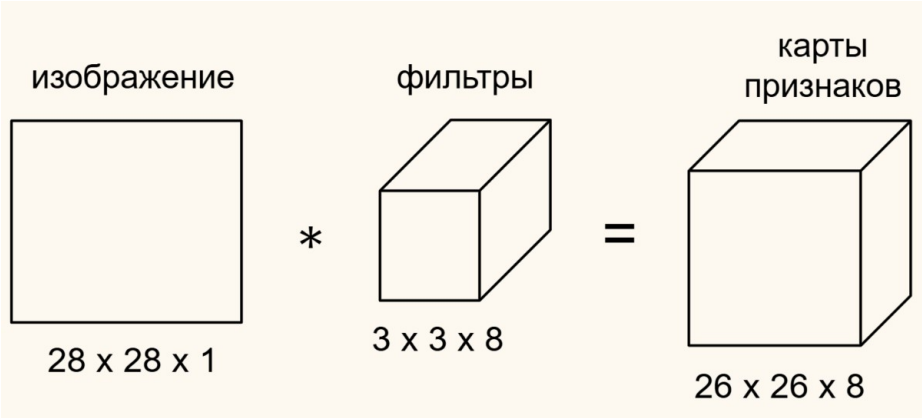


В самом деле, перемещая фильтр с размером шага 1, мы «съедаем» крайние строки и столбцы, тем самым уменьшая оба измерения на два.

Пойдем дальше. Мы уже отметили, что цифры состоят из контуров, причем эти контуры не обязательно вертикальные или горизонтальные, поэтому, скорее всего, нам понадобится больше двух фильтров. Пусть всего фильтров будет восемь.

> Заметим, что количество фильтров свёрточного слоя, а также размер шага и отступ — это гиперпараметры, которые настраиваются при обучении модели.

В этом случае мы будем производить свёртку картинки с каждым из восьми фильтров и на выходе ожидаемо получим восемь карт признаков размером 26 x 26.



Увеличим размерность каждой картинки с двух до трех измерений, то есть 28 x 28 x 1. Это требование библиотеки Keras, и зачем это нужно, станет понятно чуть позже.

```
# параметр axis функции np.expand_dims() указывает,
# где именно разместить новое измерение
X_train = np.expand_dims(X_train, axis = -1)
X_test = np.expand_dims(X_test, axis = -1)
```

```
X_train.shape, X_test.shape
```

```
((60000, 28, 28, 1), (10000, 28, 28, 1))
```

Про количество параметров

Как мы сказали выше, весами в свёрточном слое являются значения фильтров. В нашем случае их 3 x 3 x 8 = 72.

Обратим внимание на важный момент. Количество весов зависит только от размера и количества фильтров, но никак не от размера изображения. Другими словами, мы можем добавить еще один фильтр, и тогда количество весов увеличится до 3 x 3 x 9 = 91, однако если мы возьмем изображение с большей размерностью, скажем 900 x 900 пикселей, количество параметров будет по-прежнему равно 72.

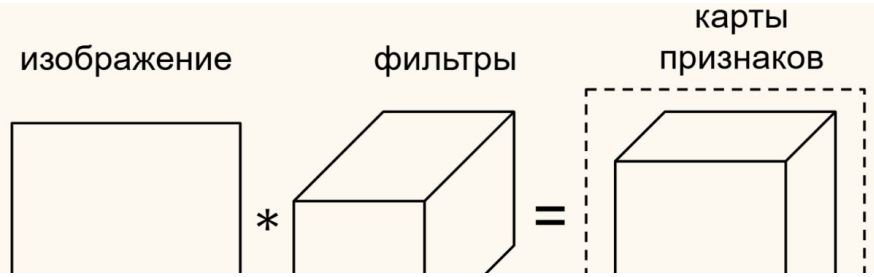
Это свойство свёрточной сети существенно снижает вычислительную затратность обучения модели, а также её склонность к переобучению.

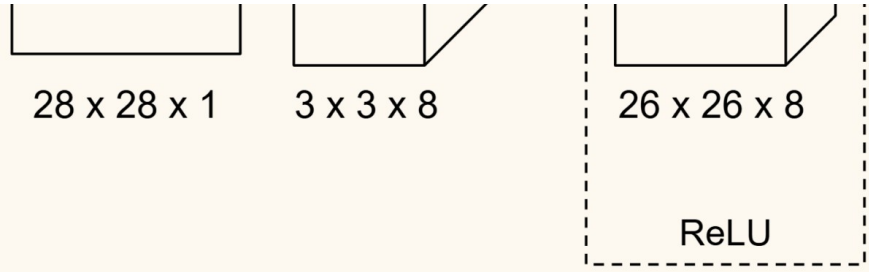
Функция активации

Результат свёртки, то есть каждое значение тензора карты признаков 26 x 26 x 8 необходимо «пропустить» через функцию активации, чтобы добавить нелинейности нашей функции.

Довольно часто в качестве функции активации используется уже знакомая нам функция **ReLU** или Rectified Linear Unit.

$$f(x) = \max(0, x)$$



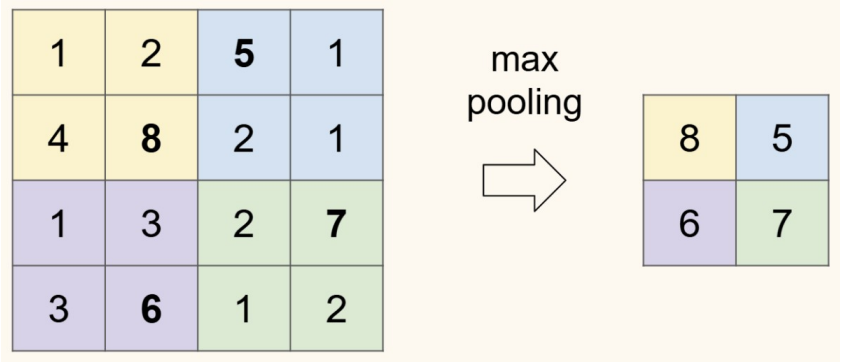


Примечание. Прежде чем передавать значения карты признаков в функцию активации, мы могли бы добавить **смещение** (bias), однако пока для простоты не будем этого делать.

Операция подвыборки

После свёрточного слоя возникает желание понизить размерность получившегося тензора. Для этой цели применяется **операция подвыборки** (pooling, а также subsampling или downsampling operation). Сам слой при этом называется слоем **подвыборки** (pooling layer).

Рассмотрим несложный пример. Пусть получившаяся карта признаков имеет размерность 4 x 4, и нам бы хотелось сократить размерность в два раза до 2 x 2. Мы могли бы разделить изображение на квадраты, а затем взять из каждого квадрата только максимальное значение.

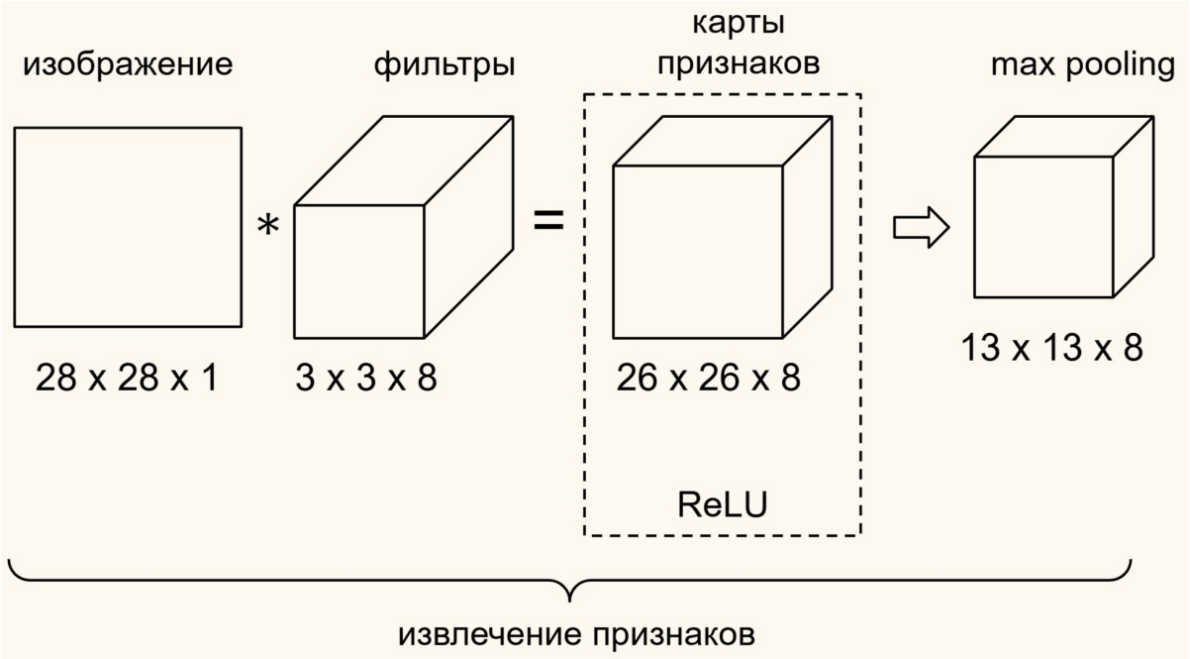


Такая операция называется **подвыборкой по максимальному значению** (max pooling). Идея, помимо понижения размерности, заключается в том, чтобы сохранить высокие значения (ведь высокие значения означают, что признак, например, контур, обнаружен) и

отбросить низкие.

Заметим, что в примере выше итоговая размерность после подвыборки по максимальному значению совпадает с операцией свёртки с размерностью фильтра 2 x 2 и размером шага 2. Это знание пригодится нам чуть позже, когда мы будем собирать модель в Keras.

Применим max pooling к картам признаков создаваемой нами нейросети. *Каждая* карта признаков, таким образом, сократится с 26 x 26 до 13 x 13 пикселей.



Дополнительно заметим, что параметры слоя подвыборки не участвуют в обучении модели (расчете обратного распространения ошибки). Их называют **необучаемыми** (non-trainable) параметрами, в отличие **обучаемых** (trainable) параметров свёрточного слоя.

Посмотрим, что было сделано до сих пор:

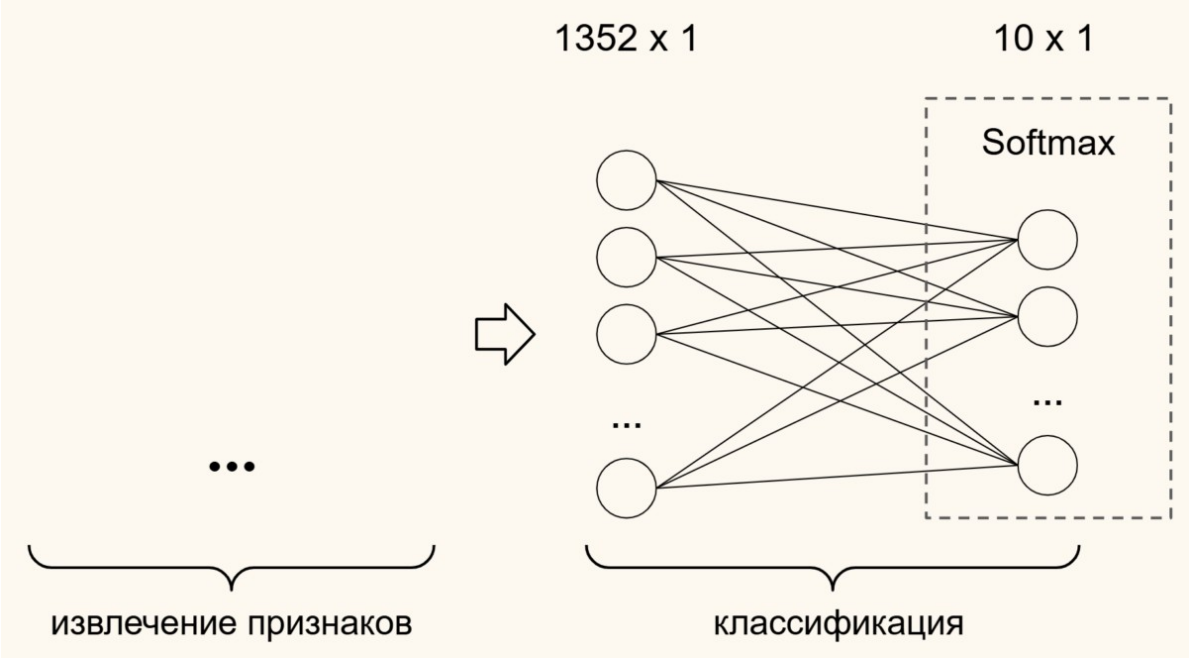
- мы применили фильтры вначале со случайными, а затем, после обучения, более оптимальными значениями;
- обнаружили признаки; и
- сократили размерность карт признаков с помощью max pooling.

Этот этап называется **извлечением признаков** (feature extraction). Теперь нам хотелось бы сделать прогноз, предсказать, о какой цифре идет речь, то есть перейти к этапу **классификации** (classification).

«Вытягивание» карт признаков. Для того чтобы двигаться дальше превратим трехмерный тензор 13 x 13 x 8 в вектор (одномерный массив) из 1 352 значений. За такую операцию будет отвечать специальный слой, который в библиотеке Keras называется Flatten.

Полносвязный слой

Наконец, получившийся вектор из 1 352 признаков мы соединим с выходным полносвязным слоем из 10 нейронов, по одному для каждого из десяти предсказываемых классов, и пропустим через функцию softmax (смещение опять же пока применять не будем).



По сути здесь мы используем уже знакомую нам полносвязную нейросеть с тем отличием, что у нас остается только входной слой из 1 352 признаков и выходной слой из 10 нейронов. Несложно посчитать, что для полносвязного слоя нам нужно обучить 10 x 1 352 = 13 520 параметров, связывающих каждый нейрон слоя Flatten с каждым

нейроном выходного слоя.

Примечание. Здесь будет уместно пояснить, что карта признаков называется так ровно потому, что получившиеся 1 352 значения являются (промежуточными) признаками для дальнейшего обучения классификатора.

Создание и обучение модели

Мы готовы создать модель в Keras. Приведем код с пояснениями.

```
model = keras.models.Sequential([
  keras.layers.InputLayer(shape = (28, 28, 1)),
  keras.layers.Conv2D(filters = 8, # количество фильтров
                      kernel_size = 3, # размер фильтра
                      strides = 1, # размер шага
                      padding = 'valid', # тип отступа
                      activation = 'relu', # активация свёрточного
                      use_bias = False), # применение смещения
  keras.layers.MaxPooling2D(pool_size = 2), # размер окна max pool
  keras.layers.Flatten(),
  keras.layers.Dense(units = 10, # количество нейронов выходного с
                     activation = 'softmax', # активация выходного
                     use_bias = False,) # применение смещения
])

model.compile(
  optimizer = 'adam',
  loss = 'categorical_crossentropy',
  metrics = ['accuracy'],
)

model.fit(
  X_train,
  keras.utils.to_categorical(y_train),
  epochs = 3,
  # будем сразу оценивать качество на тестовых данных
  validation_data = (X_test, keras.utils.to_categorical(y_test)),
)
```

```
Epoch 1/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 21s 11ms/step - accuracy: 0.8504 -
```

```
Epoch 2/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 19s 10ms/step - accuracy: 0.9673 -
Epoch 3/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 19s 10ms/step - accuracy: 0.9772 -
<keras.src.callbacks.history.History at 0x7cfa41528250>
```

```
model.summary()
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
conv2d_6 (Conv2D)	(None, 26, 26, 8)	72
max_pooling2d_5 (MaxPooling2D)	(None, 13, 13, 8)	0
flatten_3 (Flatten)	(None, 1352)	0
dense_6 (Dense)	(None, 10)	13,520

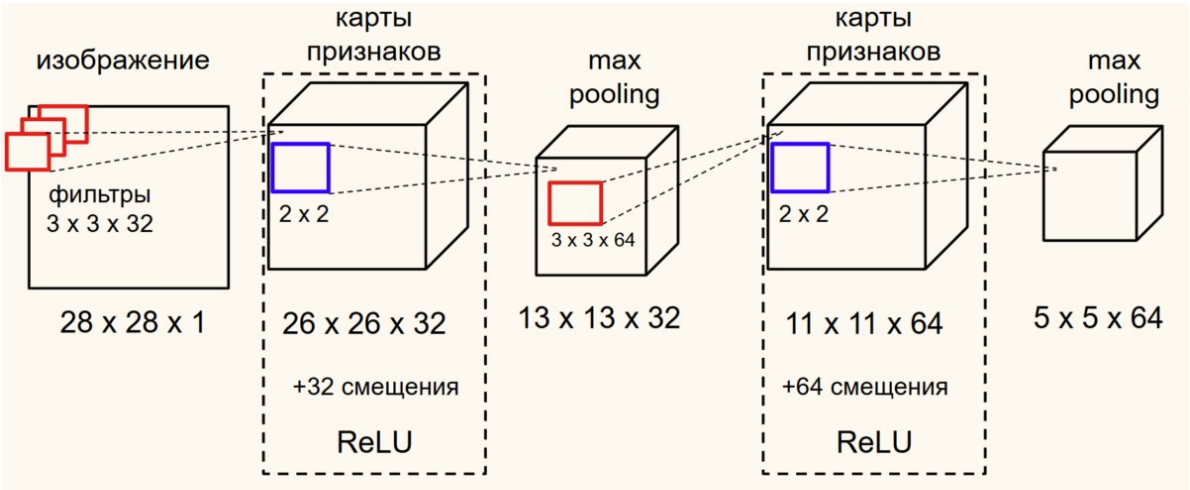
Total params: 40,778 (159.29 KB)
Trainable params: 13,592 (53.09 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 27,186 (106.20 KB)

Теперь посмотрим, как улучшить полученный результат.

Улучшение базовой модели

Количество свёрточных слоёв

Очевидным способом улучшить качество нейросети является увеличение количества свёрточных слоёв. Попробуем применить два слоя, причем в первом используем 32 фильтра, а во втором — 64.



1-ый свёрточный слой 2-ой свёрточный слой

Выбор меньшего количества фильтров для первого слоя и большего для второго не случаен. Первый слой запоминает отдельные фрагменты изображений (контуры), второй и последующие слои собирают из этих фрагментов все более крупные части изображения.

> Также говорят, что алгоритм учится создавать **представления** (Representation Learning) данных (например, изображений или текста) из более простых (низкоуровневых) составляющих.

Приведем упрощенный пример работы нейросети, первый слой которой учится определять фрагменты лиц, а второй слой — уже сами лица.

1-ый свёрточный слой (отдельные фрагменты) 2-ой свёрточный слой (комбинации фрагментов)

Вполне очевидно, что отдельных фрагментов всегда меньше, чем их комбинаций. Например, из двух фрагментов можно составить $2^2 = 4$ комбинации. Именно поэтому мы используем меньшее количество фильтров на первых свёрточных слоях.

Теперь поясним приведенную выше схему сети. **Первый свёрточный слой** использует 32 фильтра размером 3 x 3, а также 32 значения смещений по одному для каждого фильтра (значение смещения соответствующего фильтра поэлементно прибавляется к каждому

значению фильтра с помощью трансляции (broadcasting)).

Таким образом, получается 3 x 3 x 32 + 32 = 320 обучаемых параметров. Все остальные характеристики первого слоя, то есть опять же размер фильтра 3 x 3, размер окна max pooling 2 x 2, а также получающиеся размерности карты признаков 26 x 26 и операции подвыборки 13 x 13 остаются без изменений.

На **втором свёрточном слое** мы используем уже 64 фильтра 3 x 3, которые будучи применёнными к каждому из 32 слоев max pooling предыдущего слоя создают карту признаков размерностью 11 x 11 x 64 и требуют 3 x 3 x 32 x 64 = 18 432 обучаемых параметра. Вместе с 64 значениями смещения получается 18 432 + 64 = 18 496 обучаемых параметров.

Операция подвыборки второго слоя с помощью окна размерностью 2 x 2 сокращает каждую из 64 карт признаков до размера 5 x 5.

Классификация

Получившийся трехмерный тензор второго свёрточного слоя, как и в предыдущем примере, слой Flatten «вытягивает» до вектора, состоящего из 5 x 5 x 64 = 1 600 значений.

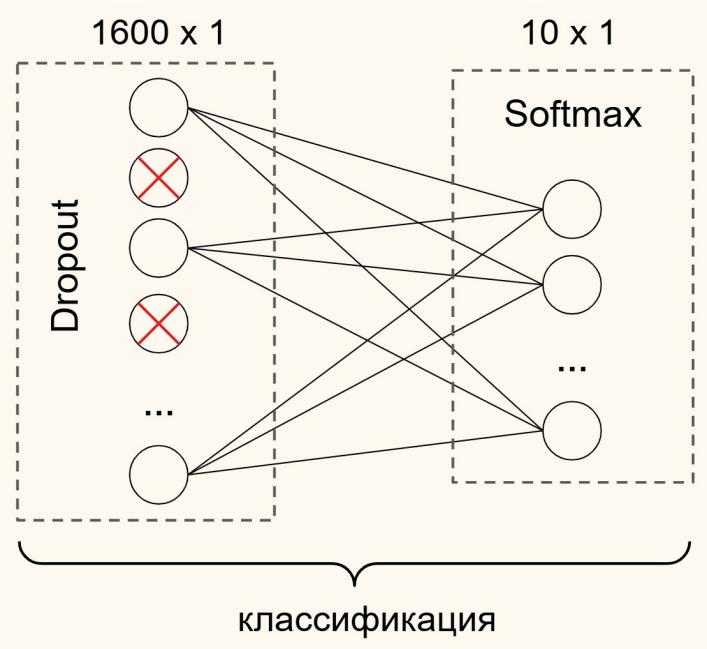
Если бы мы сразу, как и в прошлый раз, передали эти значения десяти нейронам выходного слоя, то у нас получилось бы (с учетом 10-ти параметров смещения) 16 000 x 10 + 10 = 16 010 обучаемых значений. Опять же, такое количество параметров может иметь склонность к переобучению.

Метод Dropout

В качестве метода регуляризации нейросети используем технику

Dropout («исключение», «выпадение»). Ее суть заключается в том, чтобы в процессе стохастического градиентного спуска в каждой mini batch исключать/отбрасывать определенный процент p нейронов, обнуляя их значения.

Таким образом, эти нейроны не участвуют в прямом распространении на данной итерации и также не учитываются при расчете градиентов и обновлении весов. На следующей итерации при обучении на новой mini batch исключаются другие нейроны (говорят, что используется новая маска (mask)).



Заметим, что на этапе предсказания Dropout не применяется (в Keras «отключение» этого слоя происходит автоматически). Поэтому для того чтобы на тестовых данных не было искажений, в процессе Dropout значения неисключенных нейронов умножаются на

$$\frac{1}{1-p},$$

где p — вероятность исключения одного нейрона (такой метод называют **Inverted Dropout**). Приведем пример. Пусть некоторый слой нейросети состоит из шести нейронов, а вероятность p равна $0{,}5$. Тогда каждый из трех неисключенных нейронов необходимо умножить

на $\frac{1}{1-0{,}5} = 2$.

Нейроны	1	2	3	4	5	6	Сумма
Значения до исключения	2,33	1,28	0,15	1,74	2,29	3,02	10,81
Исключенные значения	0	1,28	0	1,74	2,29	0	5,31
Значения после умножения на 2	0	2,56	0	3,48	4,58	0	10,62

Приведем ссылку на оригинальную статью по методу Dropout.

Создание и обучение модели

Напишем код модели в Keras. Вначале зададим архитектуру.

```
model2 = keras.Sequential(
    [
        keras.layers.InputLayer(shape = (28, 28, 1)),
        keras.layers.Conv2D(filters = 32,
                            # размер фильтра можно задать через ко
                            # указывающий на размерность по каждому
                            kernel_size = (3, 3),
                            activation = 'relu',
                            use_bias = True),
        keras.layers.MaxPooling2D(pool_size = (2, 2)),
        keras.layers.Conv2D(filters = 64,
                            kernel_size = (3, 3),
                            activation = 'relu',
                            use_bias = True),
        keras.layers.MaxPooling2D(pool_size = (2, 2)),
        keras.layers.Flatten(),
        # вероятность исключения нейрона задается через rate
        keras.layers.Dropout(rate = 0.5),
        keras.layers.Dense(10, activation = 'softmax'),
    ]
)
```

Соберем и обучим модель.

```
model2.compile(
  optimizer = 'adam',
  loss='categorical_crossentropy',
  metrics=['accuracy'],
)

model2.fit(
  X_train,
  keras.utils.to_categorical(y_train),
  batch_size = 64, # увеличим размер одной batch
  epochs = 10,
  validation_split = 0.1
)
```

```
Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 44s 50ms/step - accuracy: 0.8231 - los
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 82s 50ms/step - accuracy: 0.9719 - los
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 82s 49ms/step - accuracy: 0.9774 - los
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 82s 50ms/step - accuracy: 0.9813 - los
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 81s 49ms/step - accuracy: 0.9824 - los
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 83s 50ms/step - accuracy: 0.9859 - los
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 82s 50ms/step - accuracy: 0.9869 - los
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 41s 48ms/step - accuracy: 0.9870 - los
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 42s 49ms/step - accuracy: 0.9873 - los
Epoch 10/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 41s 49ms/step - accuracy: 0.9873 - los
<keras.src.callbacks.history.History at 0x7cfa415ce290>
```

Оценим качество модели на тестовых данных.

```
print("Test accuracy:", model2.evaluate(X_test,
                                        keras.utils.to_categorical
                                        verbose = 0)[1])
```

```
Test accuracy: 0.9918000102043152
```

Также обратите внимание, в первом примере мы использовали разделение на обучающую и тестовую выборки, а во втором — на обучающие, валидационные и тестовые данные. Посмотрим на количество параметров.

```
model2.summary()
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
conv2d_7 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_6 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_8 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_7 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten_4 (Flatten)	(None, 1600)	0
dropout_4 (Dropout)	(None, 1600)	0
dense_7 (Dense)	(None, 10)	16,010

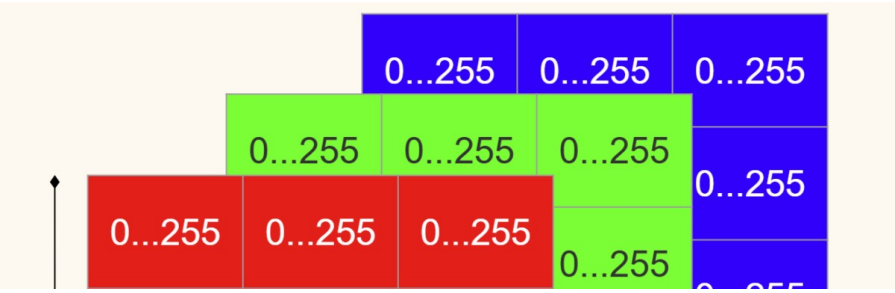
Total params: 104,480 (408.13 KB)
Trainable params: 34,826 (136.04 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 69,654 (272.09 KB)

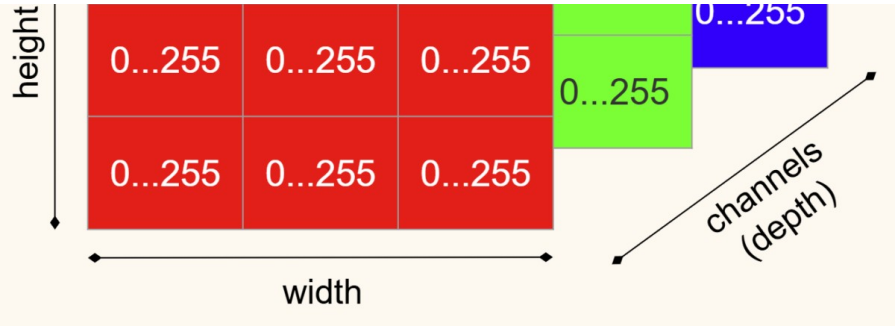
Перейдем к работе с цветными изображениями.

Работа с цветными изобрежениями

Свёртка по объёму

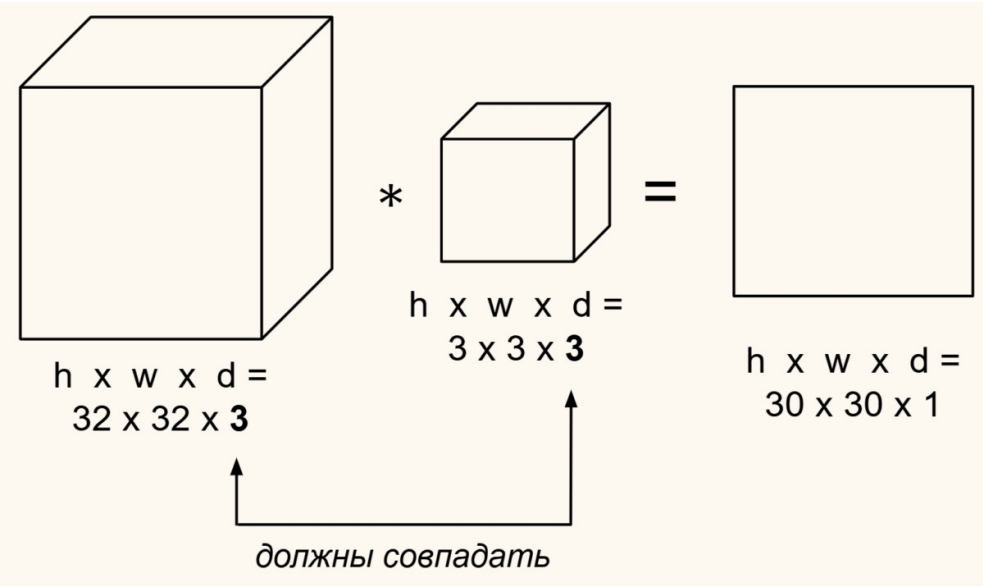
Особенность работы с цветными изображениями заключается в том, что теперь у нас есть три матрицы для красного, зеленого и синего **каналов** (RGB channels).



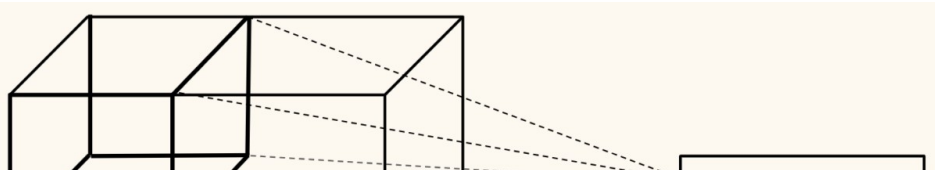


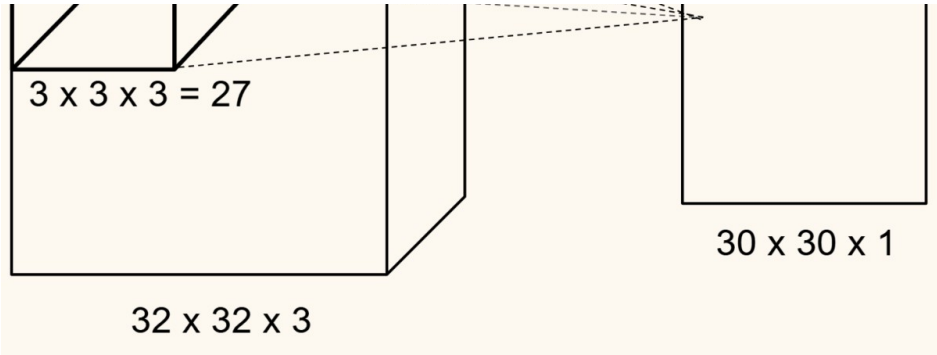
Как следствие, принято говорить о **высоте** (height), **ширине** (width) и **глубине** (depth) входных данных. Именно по этой причине мы превращали данные MNIST из матрицы в трехмерный тензор 28 x 28 x 1. Черно-белые изображения рассматриваются как частный случай цветных изображений с одним каналом (глубиной 1).

Свёртка, называемая в этом случае **свёрткой по объёму** (convolution over volume), будет происходить с также трехмерным фильтром, глубина которого должна совпадать с глубиной изображения.



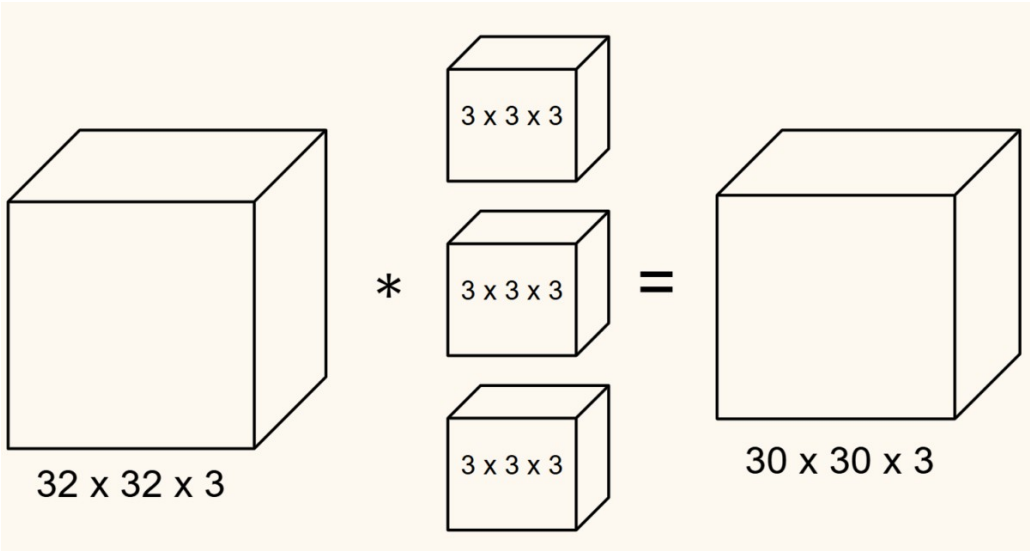
Технически, наложив трехмерный фильтр на, скажем, левый верхний угол тензора изображения, мы поэлементно перемножим каждое значение изображения с каждым значением фильтра и сложим произведения.





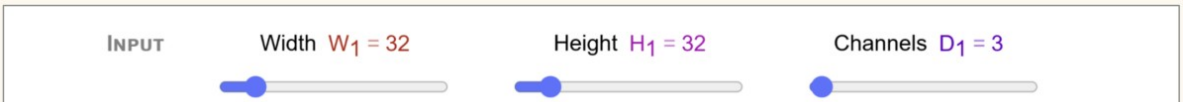
Например, если фильтр имеет размерность 3 x 3 x 3 мы поэлементно перемножим каждый из 27 элементов выбранного сегмента изображения с соответствующим элементом фильтра и сложим результаты. Эта операция и даст нам один элемент карты признаков. Несложно увидеть, что свёртка по объёму превращает трехмерные объекты в двумерные.

В случае нескольких фильтров каждый из них создаст двумерную карту признаков, которые вместе сформируют трехмерный тензор.



Калькулятор

Убедимся в правильности расчетов с помощью онлайн-калькулятора.



Немного терминологии. Поясним, что размер фильтра в этом калькуляторе обозначен через термин «**пространственные границы**» (spatial extent). Также для характеристики площади в пикселях, покрываемой фильтром, говорят о **рецептивном поле** (receptive field) нейрона.

CIFAR-10

Перейдем к созданию уже немного более сложной сети-классификатора цветных изображений из датасета CIFAR-10.

Этот набор данных содержит 60 000 изображений размером 32 x 32 x 3, на которых представлены объекты следующих десяти классов.

Класс	Объект	Класс	Объект
0	Самолет	5	Собака
1	Автомобиль	6	Лягушка
2	Птица	7	Лошадь
3	Кошка	8	Корабль
4	Олень	9	Грузовик

Импортируем библиотеку и посмотрим на первые изображения.

```
(X_train, y_train), (X_test, y_test) = keras.datasets.cifar10.load
```

```
X_train.shape, X_test.shape
```

```
((50000, 32, 32, 3), (10000, 32, 32, 3))
```

```
y_train.shape, y_test.shape
```

```
((50000, 1), (10000, 1))
```

```
labels = ['airplane', 'automobile', 'bird', 'cat', 'deer',
          'dog', 'frog', 'horse', 'ship', 'truck']

fig, axes = plt.subplots(nrows = 2, ncols = 5, figsize = (8, 4))
axes = axes.ravel()
for i in np.arange(0, 10):
    axes[i].imshow(X_train[i])
    axes[i].axis('off')
    axes[i].set_title(labels[np.squeeze(y_train)[i]], fontsize = 1

plt.subplots_adjust(wspace = 0.3)
```

Примечание. Подробный разбор применяемых для визуализации изображений функций и методов можно найти здесь и здесь.

```
X_train = (X_train / 255) - 0.5
X_test = (X_test / 255) - 0.5
```

Batch Normalization (BN)

Одной из сложностей, возникающих при обучении нейросети является так называемый **внутренний ковариационный сдвиг** (internal covariate shift), явление при котором от одной итерации к другой значения нейронов данного слоя могут существенно изменяться в зависимости от используемых данных (batches), что (1) дестабилизирует обучение (усложняет сходимость) и в конечном итоге (2) негативно сказывается на обобщающей способности алгоритма.

Как следствие, возникла идея на этапе **обучения** для *каждой карты признаков* (каждого канала) нормализовать (или, говоря более точно, стандартизовать) значения batch B, т.е. привести распределение к нулевому среднему и единичному СКО с помощью выборочных среднего $\hat{\mu}_B$ и СКО $\hat{\sigma}_B$.

При этом для того чтобы приблизить нормализованное распределение к фактическому, результат «восстанавливается», т.е. масштабируется и сдвигается на величину двух обучаемых параметров γ и β.

$$B_{\text{norm}} = \gamma \odot \frac{B - \hat{\mu}_B}{\hat{\sigma}_B + \varepsilon} + \beta$$

где ε — небольшая константа, предотвращающая деление на ноль, а $\odot$ — произведение Адамара. Таким образом, для того чтобы нормализовать тензор размерностью, например, 28 x 28 x 64 потребуется два необучаемых $\hat{\mu}_B$, $\hat{\sigma}_B$ и два обучаемых γ, β параметра для каждого из 64 каналов или 4 x 64 = 256 параметров.

На этапе предсказания данные нормализуются через скользящее среднее и скользящее СКО, полученные при обучении, а затем преобразуются с помощью выученных значений γ и β.

Важным вопросом является выбор этапа, на котором следует применять batch normalization. Авторы оригинальной статьи, предложившие этот метод, указывают, что преобразование данных следует использовать после операции свёртки и до применения функции активации (например, ReLU).

При этом многие исследователи и, в частности, создатель Keras Франсуа Шолле (François Chollet) говорят о том, что BN стоит использовать уже после ReLU.

Перейдем к созданию нейросети для классификации изображений из CIFAR-10.

Архитектура сети

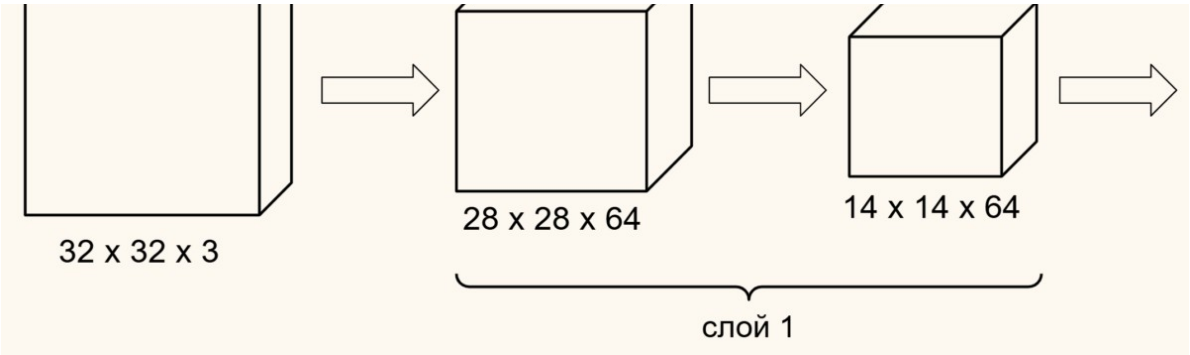
Сеть будет состоять из трёх свёрточных слоёв, причём первый и третий слои будут состоять из свёртки, batch normalization и подвыборки, а второй слой — только из свёртки и batch normalization.

Примечание. Здесь стоит добавить, что при обсуждении архитектуры CNN, слоями называют как совокупность, например, свёрточного слоя и слоя подвыборки, так и отдельно свёрточный слой или слой max pooling.

Зададим размерности тензоров и рассчитаем количество параметров нейросети. Начнем со свёрточных слоёв. Размер входного изображения составляет 32 x 32 x 3 и при размере одного из 64 фильтров 5 x 5 x 3 результатом первого свёрточного слоя будет тензор признаков 28 x 28 x 64 (размер шага 1).

При этом потребуется 5 x 5 x 3 x 64 + 64 = 4 864 параметра свёртки и 4 x 64 = 256 параметров batch normalization. Еще раз обратим внимание, что только половина из них, то есть 256 / 2 = 128 являются обучаемыми.

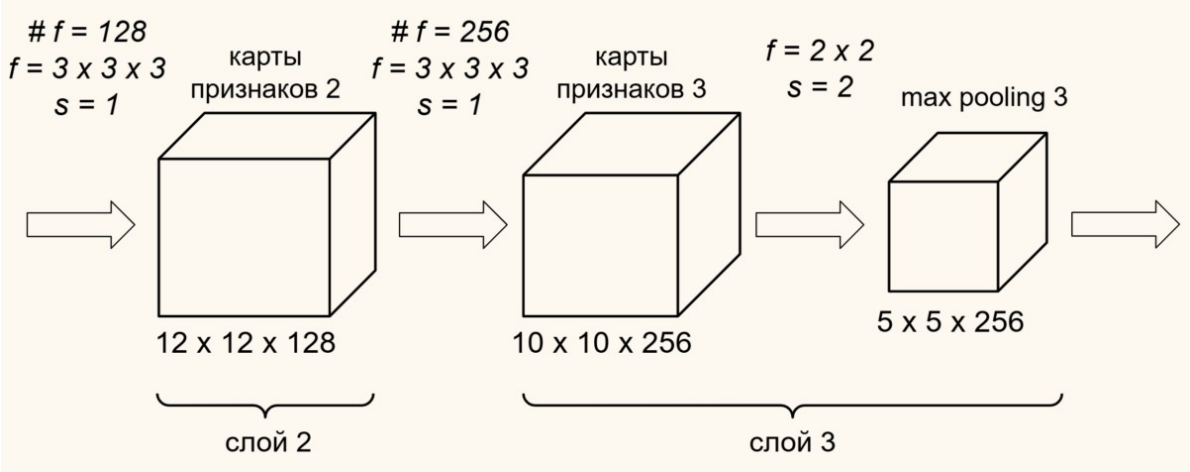
Результатом max pooling (размер окна 2 x 2, шаг 2) будет тензор 14 x 14 x 64.



Поясним, что #f — количество фильтров, f — размер фильтра, s — stride, то есть размер шага.

Во втором свёрточном слое при размере одного из 128 фильтров 3 x 3 x 3 результатом первого сверточного слоя будет тензор признаков 12 x 12 x 128 (размер шага 1). Для свёртки потребуется 3 x 3 x 64 x 128 + 128 = 73 856 параметров, для BN — 4 x 128 = 512 параметров. Max pooling здесь не применяется.

В третьем слое будет уже 256 фильтров 3 x 3 x 3 и результатом станет тензор 10 x 10 x 256 (размер шага 1). Потребуется 3 x 3 x 128 x 256 + 256 = 295 168 параметров свёртки и 4 x 128 = 1 024 BN. После max pooling (размер окна 2 x 2, шаг 2) получим тензор 5 x 5 x 256.



Таким образом, общее количество параметров свёртки составит 4 864 + 73 856 + 295 168 = 373 888, а параметров BN 256 + 512 + 1 024 = 1 792, из которых половина или 896 будут необучаемыми.

Перейдем к полносвязным слоям. Слой Flatten состоит из 6 400

нейронов, первый полносвязный слой — из 1 024. Количество весов 6 400 x 1 024 + 1 024 = 6 554 624. Между последующими слоями (см. схему ниже) потребуется 1 024 x 1 024 + 1 024 = 1 049 600, 1 024 x 512 + 512 = 524 800 и наконец 512 x 10 + 10 = 5 130 параметров.

Таким образом, в полносвязных слоях будет 6 554 624 + 1 049 600 + 524 800 + 5 130 = 8 134 154 параметров. Общее же количество обучаемых параметров составит 373 888 + 896 + 8 134 154 = 8 508 938.

Также заметим, что существуют более эффективные (и более сложные) архитектуры свёрточных сетей. Нашей целью сегодня было познакомиться с основными инструментами таких моделей и продемонстрировать общие принципы построения CNN.

Создание и обучение модели

Зададим архитектуру в Keras.

```
model3 = keras.Sequential(
    [
        keras.layers.InputLayer(shape = (32, 32, 3)),

        keras.layers.Conv2D(filters = 64,
                            kernel_size = (5, 5),
                            strides = (1, 1),
                            padding = 'valid',
                            activation = 'relu',
                            use_bias = True),
```

```
        keras.layers.BatchNormalization(),
        keras.layers.MaxPooling2D(pool_size = (2, 2),
                                  strides = 2),

        keras.layers.Conv2D(filters = 128,
                            kernel_size = (3, 3),
                            strides = (1, 1),
                            padding = 'valid',
                            activation = 'relu',
                            use_bias = True),
        keras.layers.BatchNormalization(),

        keras.layers.Conv2D(filters = 256,
                            kernel_size = (3, 3),
                            strides = (1, 1),
                            padding = 'valid',
                            activation = 'relu',
                            use_bias = True),
        keras.layers.BatchNormalization(),
        keras.layers.MaxPooling2D(pool_size = (2, 2),
                                  strides = 2),

        keras.layers.Flatten(),
        keras.layers.Dropout(rate = 0.5),
        keras.layers.Dense(1024, activation = 'relu'),
        keras.layers.Dropout(rate = 0.5),
        keras.layers.Dense(1024, activation = 'relu'),
        keras.layers.Dropout(rate = 0.5),
        keras.layers.Dense(512, activation = 'relu'),

        keras.layers.Dense(10, activation = 'softmax'),
    ]
)
```

Соберём и обучим модель.

```
model3.compile(
  optimizer = keras.optimizers.Adam(learning_rate = 0.001),
  loss = 'categorical_crossentropy',
  metrics = ['accuracy'],
)

model3.fit(
  X_train,
  keras.utils.to_categorical(y_train),
  batch_size = 64,
```

```
  epochs = 10,
  validation_split = 0.1
)
```

```
Epoch 1/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 477s 666ms/step - accuracy: 0.3067 -
Epoch 2/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 499s 662ms/step - accuracy: 0.5228 -
Epoch 3/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 496s 654ms/step - accuracy: 0.6158 -
Epoch 4/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 503s 656ms/step - accuracy: 0.6705 -
Epoch 5/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 505s 661ms/step - accuracy: 0.7114 -
Epoch 6/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 500s 658ms/step - accuracy: 0.7418 -
Epoch 7/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 494s 647ms/step - accuracy: 0.7656 -
Epoch 8/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 513s 663ms/step - accuracy: 0.7817 -
Epoch 9/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 499s 659ms/step - accuracy: 0.7973 -
Epoch 10/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 498s 653ms/step - accuracy: 0.8149 -
<keras.src.callbacks.history.History at 0x7cfa55143910>
```

```
print("Test accuracy:", model3.evaluate(X_test,
                                        keras.utils.to_categorical
                                        verbose = 0)[1])
```

```
Test accuracy: 0.7821999788284302
```

Дополнительно сверимся с правильностью расчетов параметров модели.

```
model3.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 28, 28, 64)	4,864
batch_normalization (BatchNormalization)	(None, 28, 28, 64)	256
max_pooling2d_3 (MaxPooling2D)	(None, 14, 14, 64)	0
conv2d_4 (Conv2D)	(None, 12, 12, 128)	73,856
batch_normalization_1 (BatchNormalization)	(None, 12, 12, 128)	512

(BatchNormalization)		
conv2d_5 (Conv2D)	(None, 10, 10, 256)	295,168
batch_normalization_2 (BatchNormalization)	(None, 10, 10, 256)	1,024
max_pooling2d_4 (MaxPooling2D)	(None, 5, 5, 256)	0
flatten_2 (Flatten)	(None, 6400)	0
dropout_1 (Dropout)	(None, 6400)	0
dense_2 (Dense)	(None, 1024)	6,554,624
dropout_2 (Dropout)	(None, 1024)	0
dense_3 (Dense)	(None, 1024)	1,049,600
dropout_3 (Dropout)	(None, 1024)	0
dense_4 (Dense)	(None, 512)	524,800
dense_5 (Dense)	(None, 10)	5,130

Total params: 25,527,712 (97.38 MB)
Trainable params: 8,508,938 (32.46 MB)
Non-trainable params: 896 (3.50 KB)
Optimizer params: 17,017,878 (64.92 MB)

Теперь создадим свёрточную нейросеть без использования Keras.